DEADLOCK DETECTION USING RESOURCE ALLOCATION GAPHS

Course Name: Operating Systems
Course Code: T7998

Submitted by:
Donthula LaxmiChandrahaas
PRN:25070721016
Team Member 2: Gokul Vinay Sohan, PRN:25070721022
Team Member 3: Dandu Vivekanand Reddy, PRN:25070721013
Team Member 4: Maddhuri Namish Narayan, PRN:25070721032

Under the guidance of:
Dr. Bibhuprasad Sahu

Department of Computer Science and Engineering
Symbiosis Institute of Technology,Hyderabad
2025-29 - Semester [III]

Abstract

In modern multitasking operating systems, processes frequently compete for limited resources (such as printers, memory blocks, or database locks). If a set of processes are blocked because each is holding a resource and waiting for another resource held by another process in the set, a condition known as a Deadlock occurs. If not handled, this freezes the system.

This project presents an interactive, web-based Deadlock Detection System built using Python. Using the Operating System concept of a Resource Allocation Graph (RAG), the system visually models processes and resources as nodes, and their interactions as directed edges. To detect deadlocks in real-time, a Depth-First Search (DFS) algorithm is implemented using the NetworkX library to continuously scan the graph for cycles (circular waits). When a cycle is detected, the system instantly highlights the deadlocked processes, alerts the user with a "System Frozen" status, and suggests resolution methods. The application, built with Streamlit, provides a highly visual and educational tool to understand how OS kernels prevent total system failure.

1.Introduction

In an Operating System, multiple processes run concurrently. To execute, these processes must request and acquire system resources. However, when resources are non-shareable (mutual exclusion), processes may end up waiting on each other indefinitely. This circular chain of dependencies is called a deadlock.
Detecting deadlocks is a critical function of an OS. A **Resource Allocation Graph (RAG)** is a directed graph used to represent the exact state of the system. By identifying cycles within this graph, the OS can determine if a deadlock has occurred. This project builds a visual simulator of the RAG to demonstrate how graph theory and DFS algorithms are applied to solve this core OS problem.

2.Problem Statement and Objectives

Problem Statement:

In complex systems, identifying which processes are deadlocked is difficult without a visual representation. If the OS cannot detect a deadlock, the involved processes will wait forever, leading to memory leaks and system unresponsiveness.

Objectives:

1. To design an interactive user interface where users can create processes and resources.
2. To dynamically build a Resource Allocation Graph based on user inputs.
3. To implement a Depth-First Search (DFS) cycle-detection algorithm to identify deadlocks.
4. To visually distinguish between a "Safe State" (Green) and a "Deadlocked State" (Red).

3.Requirements and Tools

- Hardware: Standard PC/Laptop (Minimum 4GB RAM).
- Operating System: Windows / macOS / Linux.
- Programming Language: Python 3.x
- Libraries Used:

- Streamlit (For the interactive web interface).
- NetworkX (For creating graphs and implementing the DFS cycle-detection algorithm).
- Matplotlib (For drawing the visual graph).

4.System Design/Methodology

The system applies the following OS methodology:

- Nodes:
  - Processes (P1, P2, etc.) are represented as Blue circles.
  - Resources (R1, R2, etc.) are represented as Orange circles.
- Edges(Connections):
  - Assignment Edge (R → P): Indicates a resource is currently allocated to a process.
  - Request Edge (P → R): Indicates a process is waiting for a resource.
- Algorithm: The system translates the visual graph into a mathematical Directed Graph. The nx.find_cycle() function is applied, which uses a Depth-First Search (DFS) to find a path that starts and ends at the same node. If a cycle exists (e.g., P1 → R1 → P2 → R2 → P1), a deadlock is confirmed.

5.Implementation

The application is implemented in three main modules:

1. User Interface Module: Built using Streamlit, providing a sidebar with dropdown menus for users to select a process, a resource, and the type of action (Request or Assign).
2. Graph Management Module: Uses a session state array to store edges. The NetworkX library builds the graph structure dynamically as the user clicks "Add Connection".
3. Detection & Visualization Module: Matplotlib handles the visual rendering of nodes and arrows. If the DFS algorithm detects a cycle, the specific edges involved in the deadlock are recolored to Red, and a warning is displayed on the UI.

The complete source code for this project is publicly available on GitHub at:
https://github.com/LaxmiChandrahaas/Deadlock-System-using-Resource-Allocation-Graphs

6.Testing and Results

Test Case 1:Deadlock State(Boundary/Invalid Case)

- Action: User creates a circular wait (P1 holds R1, P2 holds R2, P1 requests R2, P2 requests R1).
- Result: The DFS algorithm detects a cycle. The system status turns RED and displays "DEADLOCK DETECTED!". The edges causing the cycle are highlighted in red.

```
================================================
RUNNING: TEST CASE 1: DEADLOCK SCENARIO
================================================
Nodes: ['P1', 'R1', 'P2', 'R2']
Edges: [('P1', 'R1'), ('P1', 'R2'), ('R1', 'P1'), ('P2', 'R2'), ('P2', 'R1'), ('R2', 'P2')]
Running Depth-First Search (DFS) Cycle Detection...

[⚠️ DEADLOCK DETECTED! [⚠️
🔴 Circular Wait Path: P1 -> R1 -> P1
Status: SYSTEM FROZEN. Resolution required.
```

- Figure 1: Visualizing a circular wait causing a Deadlock.

Test Case 2:Safe State(Valid Case)

- Action: User assigns resources without creating a circular loop (P1 holds R1, P2 requests R2).
- Result: No cycles are found by the algorithm. The system status turns GREEN and

```
==================================================
RUNNING: TEST CASE 2: SAFE SCENARIO
==================================================
Nodes: ['R1', 'P1', 'R2', 'P2', 'P3', 'P4']
Edges: [('R1', 'P1'), ('R2', 'P2'), ('P3', 'R1'), ('P4', 'R2')]
Running Depth-First Search (DFS) Cycle Detection...

[✅] SYSTEM IS SAFE [✅]
Status: No circular waits detected. System is stable.
```

displays "System is SAFE".

- Figure 2: System operating in a safe state without circular dependencies.

7.Discussion

The project successfully demonstrates how graph theory is applied in Operating Systems. While resource allocation is just one part of OS memory management, the visual proof shows how quickly an algorithm can scan a system and detect a freeze. By highlighting the exact processes involved in the cycle, the OS can make an informed decision on which process to terminate (kill) to recover the system.

8.Conclusion and Future Scope

Conclusion:

This project successfully simulates a Deadlock Detection mechanism using Resource Allocation Graphs. By using DFS, the system instantly identifies circular waits and visually alerts the user, proving how OS kernels manage system stability.

Future Scope:

- Multi-Instance Resources: Currently, the graph assumes one instance per resource. The project can be upgraded to support multiple instances per resource using the Banker's Algorithm.
- Automated Recovery: Instead of just detecting the deadlock, the software could automatically suggest which process to terminate based on lowest priority

9.References

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.
2. NetworkX Documentation. Algorithms - Cycles. Retrieved from https://networkx.org/documentation/
3. Streamlit Documentation. Streamlit API Reference. Retrieved from https://docs.streamlit.io/